



Secure, Optimize, & Ship SQL+AI **Solutions**



Table of Contents

1

Module 1

Implement Data Security & Compliance with SQL

2

Module 2

Optimize Database Performance

3

Module 3

implement CI/CD with SQL Database Projects

4

Module 4

Integrate SQL Solutions with Azure Services

Module 1



Implement Data Security & Compliance with SQL

Learn to protect sensitive data and meet compliance requirements by implementing encryption, masking, access controls, and auditing across Microsoft SQL platforms.

Data Encryption Strategies

Protect data at rest and in use with the right encryption approach for each scenario.



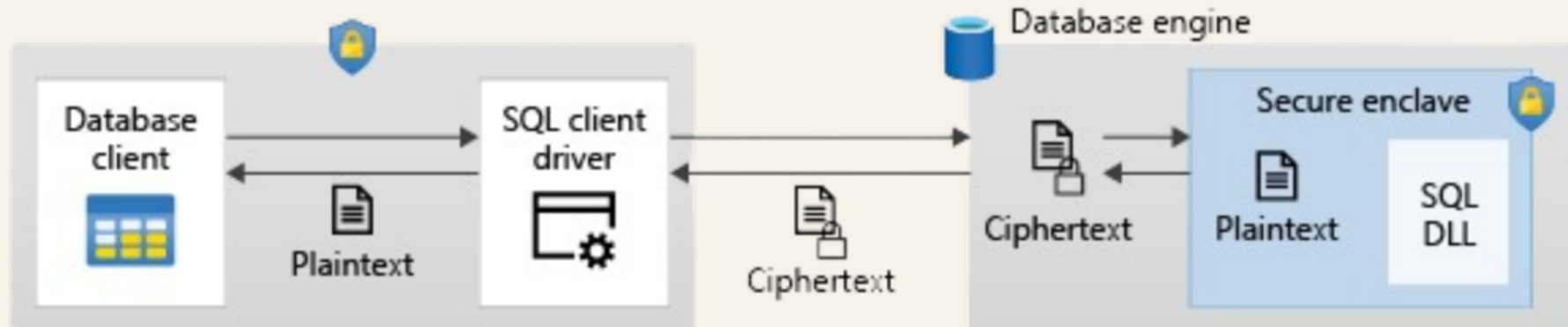
Always Encrypted

- Client-side encryption: data never exposed as plaintext on the server
- Protects data even from DBAs, sysadmins, and cloud operators
- Column Master Key (CMK) + Column Encryption Key (CEK) architecture
- Deterministic encryption: supports equality joins and GROUP BY
- Randomized encryption: stronger security, no query operations
- Secure enclaves: enable range queries and pattern matching on encrypted data
- Use cases: SSNs, credit cards, health records, financial data

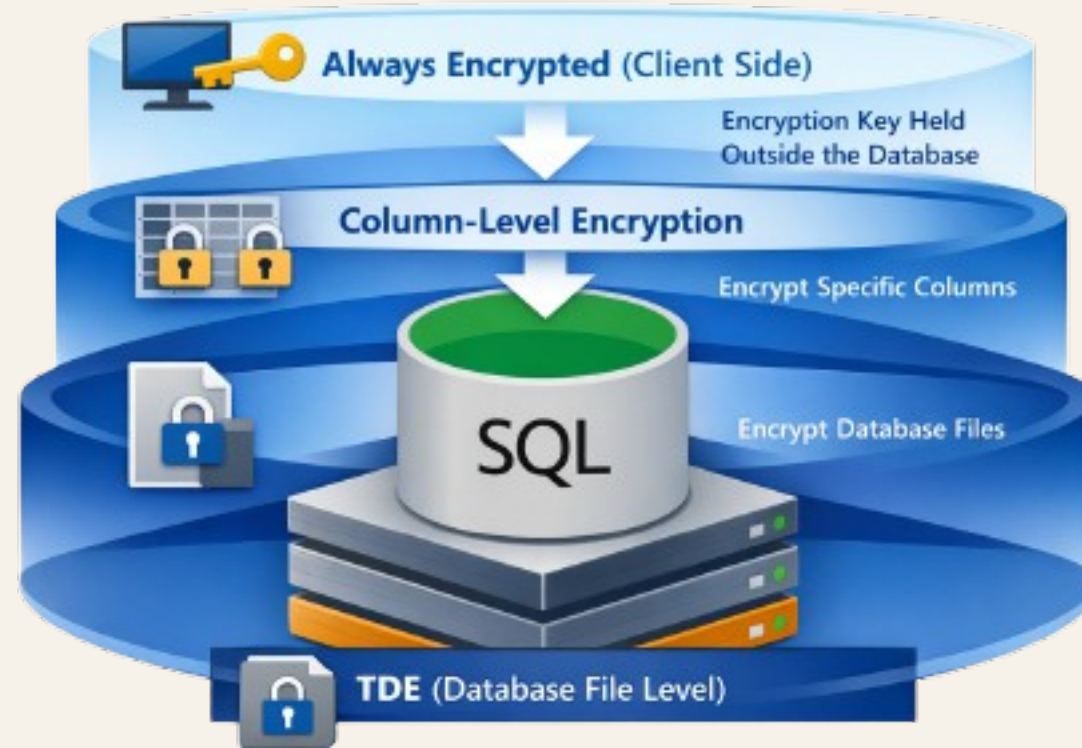


Column-Level Encryption

- Server-side encryption using symmetric keys and certificates
- Data encrypted/decrypted within the SQL Server engine
- Simpler key management than Always Encrypted
- Different trust boundary: server can see plaintext during processing
- EncryptByKey() / DecryptByKey() T-SQL functions
- Certificate hierarchy: Service Master Key > DB Master Key > Certificate > Symmetric Key
- Best for: data at rest protection when server trust is acceptable



Understand encryption layers



Dynamic Data Masking & Row-Level Security

Control what users see without changing the underlying stored data.

∞ Dynamic Data Masking (DDM)

Mask Function	Syntax	Result
Default	FUNCTION = 'default()'	xxxx or 0
Email	FUNCTION = 'email()'	aXXX@XXXX.com
Random	FUNCTION = 'random(1,100)'	Random number
Partial	FUNCTION = 'partial(2,"XX",1)'	abXXe

Important: DDM is obfuscation, not encryption. It's intended to reduce development effort by implementing Users with UNMASK permission and privileged users such as sysadmins see real data.



Row-Level Security (RLS)

Filter Predicate

Silently filters rows from SELECT, UPDATE, DELETE results

Block Predicate

Blocks INSERT or UPDATE operations that violate policy

Security Function

Inline table-valued function returning 1 (access) or 0 (deny)

Security Policy

Binds the predicate function to one or more tables

Typical pattern: SESSION_CONTEXT() or USER_NAME() in the predicate function to filter by logged-in user.

Object-Level Permissions & Secure Access

Implement least-privilege access patterns across database objects and service connections.



Permission Hierarchy

- GRANT / DENY / REVOKE at server, database, schema, and object levels
- Schema-level: GRANT SELECT ON SCHEMA::Sales TO [AnalystRole]
- Object-level: GRANT EXECUTE ON dbo.usp_GetOrders TO [AppRole]
- DENY always wins over GRANT in permission resolution
- Use database roles to group permissions, not individual user grants
- Fixed roles: db_datareader, db_datawriter, db_ddladmin
- Custom roles: CREATE ROLE [OrderProcessor] for fine-grained control
- Principle: grant minimum permissions needed for each function



Modern Secure Access

Passwordless Authentication

Microsoft Entra ID authentication instead of SQL auth with stored passwords

Managed Identity

System or user-assigned identities for service-to-service access without credentials in code

Secure Model Endpoints

Managed Identity to authenticate when calling AI model endpoints from SQL

Secure API Endpoints

Protect GraphQL, REST, and MCP endpoints with auth, authorization, and auditing

Auditing & Compliance

Capture compliance evidence with comprehensive audit trails across all SQL platforms.



SQL Server Audit

- Server-level and database-level audit specs
- Writes to Event Log or file targets
- Audit groups: BATCH_COMPLETED, LOGIN_CHANGE, SCHEMA_OBJECT_ACCESS
- Fine-grained action groups per DB object
- Combine with Extended Events for diagnostics



Azure SQL Auditing

- Built-in auditing to Blob Storage or Log Analytics
- Automatic capture of all DML and DDL ops
- Integration with Microsoft Defender for SQL
- Threat detection: SQL injection, anomalous access
- 90-day default retention, configurable to years



Compliance Features

- Data classification: sensitivity labels on columns
- Vulnerability assessment: automated security scans
- Advanced Threat Protection alerts
- Azure Policy for enforcing audit standards
- GDPR, HIPAA, SOC 2 compliance evidence

Security Decision Guide



Choose the right security control based on what you need to protect and from whom.

Requirement	Recommended Control
DBA must not see plaintext data	Always Encrypted (client-side encryption)
Encrypt specific columns, server trust OK	Column-level encryption (EncryptByKey)
Hide values from some users	Dynamic Data Masking + UNMASK permission
Each user sees only their own rows	Row-Level Security (filter predicate)
Restrict who can query tables or execute SPs	Object-level permissions (GRANT/DENY/REVOKE)
Eliminate passwords from application code	Passwordless auth + Managed Identity
Track who accessed what and when	Auditing (server & database audit specs)
Detect SQL injection and anomalous access	Microsoft Defender for SQL + Threat Detection

Exam tip: Masking is obfuscation, not encryption. If the requirement is "DBA must not see plaintext," the answer is Always Encrypted, not DDM.

Module 2



Optimize Database Performance

Choose the right service tier, manage concurrency with isolation levels, analyze queries with execution plans, and leverage Query Store for plan management.

Service Tiers & Database Configuration

Choose the right compute and storage configuration for your workload requirements.

General Purpose

- Standard compute with remote storage
- Up to 80 vCores, 400 GB memory
- Zone-redundant availability
- Best for: most business workloads

Business Critical

- Local SSD with built-in read replicas
- Lowest latency, highest IOPS
- In-memory OLTP support
- Best for: mission-critical OLTP

Hyperscale

- Distributed architecture, 128 TB+
- Near-instant backups at any size
- Up to 30 read replicas
- Best for: large, elastic workloads

Serverless

- Auto-scales compute, auto-pauses
- Pay per second of compute used
- Configurable min/max vCores
- Best for: intermittent, bursty usage

DTU Model

Bundled CPU + I/O + memory. Simple, predictable pricing. Best when you want a pre-configured compute package.

vCore Model

Independent CPU, memory, and storage scaling. Azure Hybrid Benefit eligible. Best when you need granular resource control.

Transaction Isolation Levels & Concurrency

Balance consistency and throughput by choosing the right isolation level for each workload.

Isolation Level	Dirty Read	Non-Repeatable	Phantom	Blocking
READ UNCOMMITTED	Yes	Yes	Yes	Minimal
READ COMMITTED (default)	No	Yes	Yes	Moderate
REPEATABLE READ	No	No	Yes	High
SERIALIZABLE	No	No	No	Highest
SNAPSHOT	No	No	No	None (MVCC)
READ COMMITTED SNAPSHOT	No	Yes	Yes	None (MVCC)

Azure SQL Database Default: READ COMMITTED SNAPSHOT

Azure SQL Database enables RCSI by default, using row versioning instead of locks. This eliminates reader-writer blocking but uses tempdb for the version store. SQL Server on-premises defaults to pessimistic READ COMMITTED with shared locks.

Execution Plans & Dynamic Management Views

Analyze query performance at the operator level and surface live workload metadata.

Execution Plans

Estimated Plan

Compiled without execution. Shows optimizer choices, estimated costs, and row counts.

Actual Plan

Includes actual row counts, memory grants, and runtime statistics. Essential for cardinality errors.

Key Operators to Watch

- Table/Index Scan: full scan, often a missing index signal
- Key Lookup: nonclustered index + clustered index hop
- Sort with spill: insufficient memory grant
- Implicit conversion: data type mismatch kills index seeks

Key DMVs for Performance

`sys.dm_exec_query_stats`

Cumulative CPU, reads, writes, execution counts per query plan

`sys.dm_exec_requests`

Currently executing requests with wait types and blocking info

`sys.dm_db_index_usage_stats`

Seeks, scans, lookups, updates per index since restart

`sys.dm_db_missing_index_*`

Optimizer-suggested missing indexes with estimated impact

`sys.dm_os_wait_stats`

Aggregate wait types across instance: identify bottlenecks

`sys.dm_tran_locks`

Active locks, lock modes, and resources being locked

Query Store & Query Performance Insight

Track historical query performance, detect regressions, and force optimal execution plans.

Query Store

Automatic Capture:

Stores query text, execution plans, and runtime statistics inside the database

Plan History:

Tracks all execution plans for each query over time, enabling regression detection

Plan Forcing:

Force the optimizer to use a known-good plan: `sp_query_store_force_plan`

Regressed Queries:

Built-in reports identify queries whose performance has degraded

Wait Statistics:

Query-level wait stats show exactly what each query is waiting on

Automatic Tuning:

SQL Server automatically forces last known good plan on regression

Custom Capture Policy:

Filter by execution count, CPU threshold, or stale query cleanup interval

Query Performance Insight

- Azure portal dashboard for Azure SQL Database
- Visual top resource-consuming queries
- Interactive time-range filtering
- Drill into individual query history
- Built on Query Store data
- Recommendations for indexes
- CPU, data I/O, and log I/O views

Blocking & Deadlocks

Identify and resolve concurrency problems that impact application throughput.



Blocking

Detection

- sys.dm_exec_requests: check blocking_session_id
- sys.dm_tran_locks: view lock modes and resources
- sp_who2 / Activity Monitor for quick blocking chains

Resolution Strategies

- Keep transactions short: minimize lock hold time
- Use READ COMMITTED SNAPSHOT to eliminate reader-writer blocking
- Add appropriate indexes to reduce lock duration
- Use NOLOCK hint only for monitoring (risk dirty reads)
- Configure lock timeout: SET LOCK_TIMEOUT

⇔ Deadlocks

How Deadlocks Occur

- Two+ transactions each hold a lock the other needs
- SQL Server detects cycle and kills one (the victim)

Diagnosis

- Extended Events: system_health captures deadlock graphs
- Deadlock graph XML: processes, resources, and victim
- Trace flag 1222: deadlock info in error log

Prevention

- Access objects in same order across all transactions
- Keep transactions short and at lowest isolation needed
- Use SNAPSHOT isolation to avoid lock-based conflicts

Module 3



Implement CI/CD with SQL Database Projects

Implement CI/CD for SQL Database Projects with source control, branching, schema drift detection, automated pipelines, and testing strategies.

SQL Database Projects & Source Control

Manage your database schema as code with declarative, version-controlled SQL projects.

SQL Database Projects

- Declarative model: each .sql file defines desired state of an object
- Build produces a DACPAC (data-tier application package)
- Deployment compares DACPAC to target and generates change scripts
- SDK-style project format (.sqlproj) for cross-platform tooling
- Works in VS Code, Visual Studio, and Azure Data Studio

Source Control & Branching

- Git-based version control for every schema change
- Feature branches for isolated development
- Pull requests with schema diff review before merge
- Schema comparison to sync project with live database
- Branching strategy: main > develop > feature/hotfix

Schema Drift Detection

Schema drift occurs when direct changes are made to production outside the SQL project (ad-hoc ALTER statements, manual portal changes). Detect drift by comparing the project model against the live database. Use schema comparison tools to identify untracked changes and decide whether to incorporate them into the project or revert them.

Automated Pipelines & Testing

Build, validate, test, and deploy database changes through automated CI/CD pipelines.

1

Build

Compile .sqlproj to DACPAC, validate schema references

2

Validate

Schema compare against target, review change script

3

Test

Run tSQLt unit tests, integration tests on ephemeral DB

4

Deploy

SqlPackage publish to target with pre/post scripts

Testing Strategies

- tSQLt framework for T-SQL unit tests inside the database
- Pre-deployment validation: review generated change scripts
- Post-deployment smoke tests: verify objects and data integrity
- Ephemeral databases: spin up test instances, deploy, test, tear down
- Data-driven tests: constraints, referential integrity, computed columns

Platform Support

- GitHub Actions: dotnet build + SqlPackage in CI workflows
- Azure DevOps: Azure SQL Database deployment task
- SqlPackage CLI: cross-platform publish, extract, export, import
- Pre/post-deployment scripts for data migration and seeding
- SQLCMD variables for environment-specific connection strings

Module 4



Integrate SQL Solutions with Azure Services

Create REST and GraphQL APIs with Data API Builder, deploy to Azure hosting services, and implement monitoring and event-driven patterns.

Data API Builder: REST & GraphQL APIs

Generate secure, production-ready APIs from your database schema with zero application code.

Core Capabilities

- Auto REST and GraphQL endpoint generation from tables, views, stored procs
- Configuration-driven JSON config defines entities, permissions, relationships
- Built-in authentication with Microsoft Entra ID and EasyAuth
- Role-based authorization at entity and field level
- Supports SQL Server, Azure SQL, Cosmos DB, MySQL, PostgreSQL
- Pagination, filtering, sorting, and field selection out of the box

Deployment Options

- Azure Static Web Apps (built-in database connections)
- Azure Container Apps for containerized deployment
- Azure App Service for traditional web hosting
- Local development with DAB CLI (dab start)
- Docker container for consistent dev/prod parity
- Azure API Management for advanced gateway features

Security & Performance Features

Response Caching:

Configurable per-entity cache TTL to reduce database load

Cross-Origin (CORS):

Configurable allowed origins for browser-based API access

GraphQL Depth Limiting:

Query depth limits prevent abuse and excessive resource consumption

Monitoring & Event-Driven Patterns

Implement observability and react to data changes with Azure-native integration patterns.



Azure Monitor & Diagnostics

- Azure Monitor metrics: CPU%, DTU%, storage, connections, deadlocks
- Diagnostic settings: stream to Log Analytics, Event Hubs, or Storage
- KQL queries in Log Analytics for deep performance analysis
- Alert rules: trigger notifications on threshold breaches
- Intelligent Insights: AI-detected performance anomalies



Event-Driven Change Patterns

- Change Data Capture (CDC): track row-level changes to tables
- Change Tracking: lightweight change detection for sync scenarios
- Azure Functions SQL trigger: react to changes in real-time
- Logic Apps: workflow automation from database events
- Event Hubs: stream database changes to downstream consumers

Integration Architecture Patterns

`sp_invoke_external_rest_endpoint`

Call external REST APIs directly from T-SQL (Azure SQL DB only)

Logic Apps SQL connector

Low-code workflow automation with database triggers

Azure Functions + SQL bindings

Input/output bindings for serverless database access

Event Grid + Change Events

Publish database change events for microservice architectures

Key Takeaways

- ✓ Always Encrypted protects data even from DBAs; Dynamic Data Masking is obfuscation, not encryption
- ✓ Row-Level Security filters rows transparently; combine with SESSION_CONTEXT for multi-tenant apps
- ✓ Passwordless authentication with Managed Identity eliminates credential risks in application code
- ✓ Choose service tiers based on workload: General Purpose for most, Business Critical for low-latency OLTP
- ✓ Query Store tracks historical performance and enables automatic plan regression correction
- ✓ READ COMMITTED SNAPSHOT is the Azure SQL default and eliminates reader-writer blocking
- ✓ SQL Database Projects bring declarative, source-controlled schema management with CI/CD pipelines
- ✓ Data API Builder generates secure REST and GraphQL endpoints from your database schema
- ✓ Azure Monitor, CDC, and Azure Functions SQL trigger enable observability and event-driven architectures

Source: learn.microsoft.com/training/paths/secure-optimize-deploy-database-solutions

References

Bookmark these — they cover the official exam page and the full video series.

Official DP-800 certification page

01

<https://learn.microsoft.com/en-us/credentials/certifications/developing-ai-enabled-database-solutions/?practice-assessment-type=certification>

Community series — Microsoft Reactor (Data Days: The SQL AI Series)

02

https://developer.microsoft.com/en-us/reactor/series/S-1683/?wt.mc_id=datadayslive_S-1683_email_reactor





Thank you, crew!

You've got the skills to secure, optimize, and ship production-ready SQL+AI solutions —
now go build something great.